

Smart Classroom and Timetable Scheduler: A Full-Stack Web Application with Role-Based Access Control for University Academic Management

Shamanth M*, Sanjeev G*, Shashikiran S*

*School of Computer Science and Engineering,

Presidency University, Bengaluru, Karnataka, India

Email: {SHAMANTH.20221CSE0714, SANJEEV.20221CSE0713, SHASHIKIRAN.20221CSE0785}@presidency.in

Abstract—University Course Timetabling (UCTTP) is a well known combinatorial optimization problem, and mostly accepted to be NP-hard, thus ranked among the most challenging administrative tasks of academic institutions. At Presidency University, the process of scheduling the timetable was being done manually through paper and spreadsheets, which was time consuming and highly subject to errors. These methods did not have any automated conflict detection and a situation arose where the same classroom was booked multiple times, the same teacher was assigned two or more classes with an overlap, the same or different class conflicting and institutional policies were violated which took a crucial manual approach to rectify the situation. Despite a number of algorithmic methods including heuristics, genetic algorithms and hyper-heuristics that have been suggested in the current literature, most of them do not match well with real-life institutional constraints, policies and user positions. This paper will propose and design a Smart Classroom and Timetable Scheduler, a full-stack web application that consists of React.js, and on the front end, using Spring and MySQL, as the back and data storage respectively, with secure authentication in the form of JSON Web Tokens. There are three user roles: Administrator, Teacher, and Viewer and role-based access control is implemented both at the frontend and the backend layers. The application has five pairwise constraint-based conflict detection algorithms that can detect classroom duplicates, teacher conflicts and conflicts during scheduling effectively even with large datasets. The major characteristics are an interactive weekly schedule with the possibility of drag-and-drop, alerts about the conflicts in real-time, room set-up options, workload statistic and adjustable time slots. Other limitations like holidays and exam time are also included. The system was tested in 16 test cases and it was found to have full functional accuracy in all major web browsers

Index Terms—ROLE based access control, University Course Timetabling Problem, UCTTP, JWT Authentication, Spring boot, React.js, MySQL, Conflict Detection, Margin Satisfaction, Web Application, Academic Scheduling, Single Page Application

I. INTRODUCTION

The University Course Timetabling Problem (UCTTP) can be defined as the problem to distribute a set of courses to time slots and venues available and meet a set of hard and soft constraints [1]. The literature has highly acknowledged that it is an NP-hard problem [1], [2], i.e. there are no optimal

solutions that could be assured even in the case of large-scale cases in the limited time of four Polygons.

In Presidency University, Bengaluru, timetable scheduling had always been done manually through the use of spreadsheets and use of paper. This manual process was time consuming and subject to human error. The usual problems were booked classes twice, assignments of duplicating teachers, classes clashing, and breaking the availability rules. These mistakes involved a lot of administrative work to detect and rectify them and eventually led to a lack of efficiency in operations and the quality of academics.

Even though some algorithmic solutions to UCTTP have been suggested such as meta-heuristics [2], genetic algorithms [1], simulated annealing [3], and hyper-heuristics [4], many of them are challenging to implement in real-life academic settings. They are inflexible and too complex to be practically deployed in institutions, and do not support dynamic scheduling.

To tackle these issues, the present paper proposes a Smart Classroom and Timetable Scheduler, a web-based full-stack application that is developed to be applicable in real-life academic settings. The proposed system:

- Automatically identifies scheduling issues like room conflicts, teacher collisions, overlaps of classes, availability breaches, and workload overload.
- Applies role-based access control (RBAC) through the JSON Web Token (JWT) authentication.
- Offers a real-time schedule drop-in experience with the ability to drag/drop any schedule and a conflict resolution system.
- Helps manage the rooms and teachers with configuration restrictions of capacity, availability, and workload limits.
- Also has other functions like the ability to configure time slots, holiday management, blocked out periods, CSV export and the ability to generate a printable timetable.

The rest of this paper has the following structure. Section II presents the work related literature on UCTTP and web based scheduling systems. Section III describes the constraint model and problem statement. Section IV details the system

architecture. Section V discusses the database design and key algorithms. The sixth section talks about the security structure and RBAC execution. Section VII will be a description of the features implemented. In section VIII, the evaluation and performance analysis are given. And lastly, Section IX sums up the paper and recommends future work.

II. RELATED WORK

A. University Schedule

University Course Timetabling Problem (UCTTP) is a famous mathematical programming problem that encompasses scheduling courses across time slots and across classrooms and fulfils a collection of constraints. Since it is NP-hard, it has received numerous studies in the operations research field [1].

Chen et al. [1] give a full survey of UCTTP and classify the approaches to solutions based on exact methods, meta-heuristics, and hybrid methods. In like manner, Lewis [2] examines meta-heuristic strategies and categorizes them as one-stage, two-stage and relaxation based strategies. Other articles, including those of Ceschia et al [3] and Burke et al [4], discuss state of the art methods of optimization such as simulated annealing and hyper-heuristics.

Although the approaches are very diverse, most of the current approaches are concerned more with optimization performance based on benchmark data. The major drawback is that these approaches generally impose simplistic assumptions on the underlying constraints and are not easily scaled to the realistic institutional demands like dynamic scheduling, administration process, and user interface.

B. Meta-Heuristic Approaches and Constraint-Based Approaches

The UCTTP literature is mainly dominated by constraint-based and meta-heuristic methods. Lu and Hao [5] introduced Adaptive Tabu Search (ATS) that enhances the quality of the solutions by effective search strategies. Another strategy proposed by Goh et al. [6] that merges Tabu Search and Simulated Annealing with good success on benchmark problems.

The other methods such as genetic algorithms [10], hyper-heuristics [4], and hybrid optimization models [13] have all proved useful in reducing the amount of constraint violations and enhancing the quality of timetables.

Nonetheless, there are a number of limitations to these methods. They are very computationally intensive, not easy to implement and they are not always transparent to end-users. They are also generally not made to optimize offline, and cannot handle real-time scheduling, user interaction, and incremental updates needed in the real academic world.

C. Real-World Timetabling Systems

A number of studies have tried to create the linkage between the theoretical models and practice. McCollum [7] points out the importance of difficulties in implementing optimization methods in the real world since there are different institutional constraints. Prabodanie [8] presented an integer programming

model of a real university, which proved to be feasible, but casts doubt on scalability.

Sze et al. [9] devised two-step heuristic lecture timetabling, in the absence of pre-registration information, whereas Matias et al. [10] implemented a hybrid genetic algorithm in the management of teaching workloads.

In spite of these contributions, a majority of the systems are domain-specific and are not flexible. They also usually lack simple interfaces, real-time feedback, and full-fledged management capabilities needed in the day-to-day management of the academic institution.

D. Web-Based Academic Management Systems

Recent studies have conducted studies in the area of web-based applications of scheduling systems. Vrielink et al. [11] found that there is a major divide between theoretical frameworks in timetabling and the real-world application that is executed in a web-based setting.

Some of the tools offer partial automation but they frequently lack certain features like role-based access control (RBAC) and real-time conflict detection as well as interactive user interfaces. Spreadsheet software (including Excel and Google Sheets) can be flexible, but they do not qualify as automated or validated; on the other hand, dedicated software does not tend to be customizable and may also be inaccessible.

E. Research Gap and Contribution

Based on the literature, it is visible that there is a difference between theoretical models that are optimized and deployable practical systems. A lot of methodologies either concentrate on the optimization of algorithms alone, or have less rigorous web-based solutions in place that lack strong conflict detection and security features.

To overcome these shortcomings, the given system is implemented with the constraint-based approach and the full-stack web implementation. It focuses on simplicity, usability, and real-time interaction and provides the accuracy of conflict detection. The system also combines role-based access control, interactive scheduling, and administrative characteristics, which means that it is appropriate to apply the system in the real world in any institution.

III. PROBLEM STATEMENT AND CONSTRAINT MODEL

A. Formal Definition

After the design of Chen et al [1], the UCTTP. The definition of for Presidency University would be: a set of courses C , a set of time-slots T , a set of rooms V , a set of teachers P , and a set of class sections S , find an assignment function:

$$f : C \rightarrow T \times V \times P \times S \quad (1)$$

such that all hard constraints are satisfied and soft constraint violations are minimised.

B. Constraint Model

The constraints implemented in the proposed system are presented in Table II.

TABLE I: Constraint Model of the Proposed System

ID	Constraint	Type
HC1	A teacher cannot be assigned to two classes at the same day and time	Hard
HC2	A room cannot be assigned to two classes at the same day and time	Hard
HC3	A class section cannot have two subjects at the same day and time	Hard
HC4	All lectures must be scheduled in the timetable	Hard
HC5	Pre-assigned blocked periods must be respected	Hard
HC6	A teacher can only be scheduled on their available days	Soft
HC7	A teacher's assigned hours must not exceed their maximum workload	Soft
HC8	A room under maintenance cannot be used for classes	Hard
HC9	Room capacity must accommodate the class size	Soft

C. Conflict Detection Model

The system detects five categories of conflicts, denoted $\mathcal{K} = \{K_1, K_2, K_3, K_4, K_5\}$:

- K_1 : Room double-booking — $\exists s_i, s_j \in S_{\text{scheduled}}, i \neq j$ such that $s_i.\text{room} = s_j.\text{room}$ and $s_i.\text{day} = s_j.\text{day}$ and $s_i.\text{time} = s_j.\text{time}$
- K_2 : Teacher clash — same teacher, same day, same time
- K_3 : Class double-scheduling — same section, same day, same time
- K_4 : Availability violation — teacher scheduled on unavailable day
- K_5 : Workload overload — assigned hours > maximum hours

IV. PROBLEM MODEL AND CONSTRAINT MODEL

A. Formal Definition

The UCTTP is designed after the UCTTP of Chen et al [1]. Defining Presidency University would be: there is a set of courses C , there is a set of time-slots T , there is a set of rooms V , there is a set of teachers P and there is a set of class sections S that there is an assignment function:

$$f : C \rightarrow T \times V \times P \times S \quad (2)$$

in such a way that the hard constraints are met and the violation of soft constraints is minimized.

B. Constraint Model

Table presents the constraints that will be put in place in the proposed system.

TABLE II: Constraint Model of the Proposed System

ID	Constraint Description	Type
K1	Double-booking of the room the same day, same time, same room	Hard
K2	Teacher conflict - same teacher, day, time	Hard
K3	same section, same day, same time - Class double-scheduling	Hard
K4	Availability violation - teacher is booked on day of unavailability	Soft
K5	Workload overload - number of hours assigned/maximum number of hours	Soft

C. Conflict Detection Model

On the system, five types of conflicts are identified, which are denoted.

$$K = \{K_1, K_2, K_3, K_4, K_5\}:$$

- K_1 : Double-booking of the room the same day, same time, same room.
- K_2 : Teacher conflict - same teacher, day, time.
- K_3 : same section, same day, same time - Class double-scheduling.
- K_4 : Availability violation - teacher is booked on day of unavailability.
- K_5 : Workload overload - number of hours assigned/maximum number of hours.

V. SYSTEM ARCHITECTURE

The following is the structure of the remaining part of this paper. Section II contains a comprehensive literature review of the existing literature on the topic of the University Course Timetabling Problem (UCTTP) regarding classical methods of solving problem, meta-heuristic methods and the latest web-based systems. Section III is a formal discussion of the problem statement and provides the constraint model with specifics of the hard and soft constraints a consideration in the proposed system.

Section IV presents the overall system structure, design decisions, and technology stack and interaction of the frontend with the back and database layers. Section V talks about the database design and key algorithms, and particularly, the mechanism of conflict detection and computational logic underlying the same.

Section VI is based on the implementation of security framework and role-based access control (RBAC) in terms of the authentication mechanism, as well as the authorization strategies that should be implemented to ensure the security of the system operation. Section VII proposes the main features of the system and is supported by interface level pictures and elaboration of the functionalities.

Within section VII one finds an analytic evaluation of the system, and a performance analysis, and comparison against the solutions the company has. Finally, This IX completes the paper with a conclusion summarizing the contributions, limitations, and explaining potential improvements.

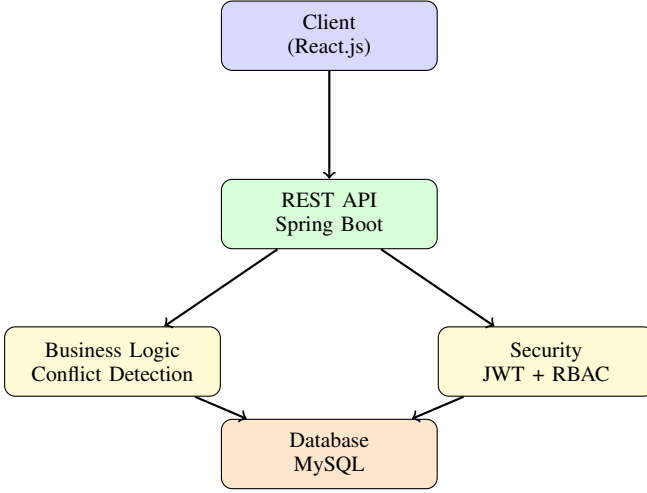


Fig. 1: System Architecture of the Smart Classroom Scheduler

A. Technology Stack

Table summarises the technology stack of each layer of application.

TABLE III: Technology Stack

Layer	Technology	Purpose
Frontend	React.js	User Interface
Backend	Spring Boot	API Development
Database	MySQL	Data Storage
Security	JWT	Authentication

B. Unidirectional Data Flow

The frontend has a unidirectional data flow pattern:

$$\begin{array}{c}
 \text{User Activity (returns API Call)} \Rightarrow \text{State Update} \\
 \Downarrow \\
 \text{UI Re-render}
 \end{array} \quad (3)$$

Global state is managed with React Context API. Owning (viewing) (creating) (refusing) (withdrawing a) to application data). All API calls get hubbed in a , with Axios interceptors, which is manually adding the JWT bearer token to every request and redirecting to the 401 Unauthorized page on the login page in case of non-Authority.

VI. DATABASE DESIGN AND KEY ALGORITHMS

DB schema was built in a way to ensure that it matches the actual world. organizing objects like teachers, rooms, time-slots, and classes. Extra precaution was taken to implement referential integrity. with foreign keys, making sure bad or non-consistent data. can not be brought into the system.

The conflict detector mechanism is the essence of the system. Instead of using advanced optimisation techniques, a pairwise The scheduling conflicts are identified by the methods of comparison. Whereas this technique has a theoretical time complexity of $O(n^2)$. it is efficient with the size of data that is usually encountered. entered in an academic environment.

A. Database Schema

The relational schema consists of 10 tables as illustrated in. Table III. Foreign key constraints are used to impose referential integrity: the slots table brings along teachers and rooms, ensuring that a planned lesson always cites up-to-date, extant entities.

TABLE IV: Database Schema Summary

Table	Key Attributes
users	id, username, password (BCrypt), role (ENUM)
rooms	id, name, capacity, type, status, notes
room_equipment	room_id (FK), equipment
teachers	id, name, subject, email, max_hours, user_id (FK)
teacher_avail	teacher_id (FK), day, available
slots	id, subject, cls, teacher_id (FK), room_id (FK), day, time_slot, color, recurring
holidays	id, date (UNIQUE), label
blocked_slots	id, day, time_slot (UNIQUE constraint)
app_settings	setting_key (UNIQUE), val
audit_log	user_id (FK), action, entity, entity_id, detail

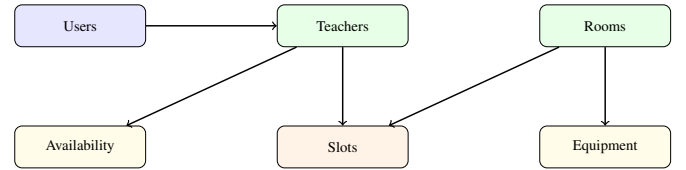


Fig. 2: Compact ER diagram of the proposed system showing relationships between users, teachers, rooms, slots, availability, and equipment.

B. Conflict Detection Algorithm

The underlying algorithm is a conflict detection engine. tribution of this system. It is a server-based service. ConflictService.java) (and represented as a mirror). client-side real-time preview. Pairwise is given in algorithm. detection procedure.

```

1: conflicts  $\leftarrow []$ 
2:  $S \leftarrow \text{SLOTREPOSITORY.FINDALL}()$ 
3: for  $i = 0$  to  $|S| - 2$  do
4:   for  $j = i + 1$  to  $|S| - 1$  do
5:      $a \leftarrow S[i]; b \leftarrow S[j]$ 
6:     if  $a.\text{day} \neq b.\text{day}$  OR  $a.\text{time} \neq b.\text{time}$  then
7:       continue
8:     end if
9:     if  $a.\text{room} = b.\text{room}$  then
10:      conflicts.add({type: "room", slots:  $[a, b]$ })
11:     end if
12:     if  $a.\text{teacher} = b.\text{teacher}$  then
13:      conflicts.add({type: "teacher", slots:  $[a, b]$ })
14:     end if
15:     if  $a.\text{cls} = b.\text{cls}$  then
16:      conflicts.add({type: "class", slots:  $[a, b]$ })
17:     end if
18:   end for
19: end for
20: for each slot  $s \in S$  do
21:   if  $s.\text{teacher.availability}[s.\text{day}] = \text{false}$  then
22:     conflicts.add({type: "availability", slots:  $[s]$ })
23:   end if
24: end for
25: for each teacher  $t$  do
26:   if  $\text{assignedHours}(t) > t.\text{maxHours}$  then
27:     conflicts.add({type: "workload", teacher:  $t$ })
28:   end if
29: end for
30: return conflicts

```

Algorithm 1: Five-Type Conflict Detection Algorithm

The time-complexity of the pair wise stage is $O(n^2)$ in which, n can be defined as the number of slots scheduled. In case of normal university scale, ($n \leq 500$), this equates to fewer than 250,000 comparisons, possible in a time of less than 10 milliseconds on modern machines. The availability and workload verification introduces $O(n)$ and $O(T)$ passes, respectively, T the number of teachers.

C. Room Utilisation Calculation

For each room r , utilisation is computed as:

$$\text{util}(r) = \frac{|\{s \in S : s.\text{room} = r\}|}{|D| \times |T|} \times 100\%$$

Fig. 3: Room Utilization Formula

where D is the set of scheduled days and T is the set of configured time-slots. A colour-coded threshold system categorises rooms as High Load ($> 70\%$), Medium Load ($40\%-70\%$), or Low Load ($< 40\%$).

D. Time-Slot Sorting Algorithm

Time slots, which administrators add, must be displayed in chronological order. As time slots become stored as strings

$$\text{toMin}(t) = \begin{cases} h \times 60 & \text{if AM and } h \neq 12 \\ 0 & \text{if AM and } h = 12 \\ (h + 12) \times 60 & \text{if PM and } h \neq 12 \\ 12 \times 60 & \text{if PM and } h = 12 \end{cases}$$

Fig. 4: Time Conversion Function

(e.g., "8:00 AM", "1:00 PM"), a comparison-based sort with a custom key function is applied:

This properly manages the 12 hour clock edge cases (mid-, $O(n \log n)$ sorts night and noon) and sort slots.

VII. SECURITY AND ROLE-BASED ACCESS CONTROL

Security: A multi-user sys is an important consideration with respect to security. tem, especially one that contains academic information. To address this, the application is JWT-based authentication, which permits. users to gain access to the system through security implementation. side sessions.

Hashing of passwords is done using BCrypt, which ensures that passwords are securely stored.even during a data breach, sensitive data is safeguarded. Every authenticated request carries a token which is whitelisted. preprocessed by the backend.

A. Authentication Flow

The system relies on stateless JWT based authentication as depicted in Fig. 4. BCrypt is used to hash passwords with a 1 work factor (1024 iterations). JWT tokens are signed contain HMAC-SHA256 and the use of a claim containing the username and role, and can be set to expire after (default 24 hours).

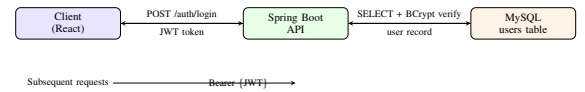


Fig. 5: JWT-based authentication flow.

B. Role-Based Access Control

There are three roles with different permissions established by the system. as in Table IV. RBAC is independently enforced on two level. dent layers: the spring security securityfilterchain. is a method-level annotation enforcer (including: @PreAuthorize). the React client conceals user interface objects and blocks routes. according to the role of the authenticated user of the JWT. payload.

TABLE V: Role-Based Access Control Matrix

Action	ADMIN	TEACHER	VIEWER
View timetable	✓	✓	✓
View conflicts	✓	✓	✓
View reports	✓	✓	✓
Export CSV / Print PDF	✓	✓	×
Add / Edit / Delete slots	✓	×	×
Manage rooms	✓	×	×
Manage teachers	✓	×	×
Settings & configuration	✓	×	×

C. API Security Rules

The `SecurityFilterChain` enforces the following HTTP-method level rules:

- GET requests on all `/api/**` endpoints require authentication and are accessible to users of any role.
- POST, PUT, and DELETE requests on resource endpoints require the ADMIN role.
- POST `/api/auth/login` is public and does not require authentication.

This makes certain that despite bypassing of the frontend UI (e.g., via direct API calls), invalid changes are rejected. and in the server with HTTP 403 Forbidden.

VIII. IMPLEMENTED FEATURES

One of the main goals of this project was to design a system which would be user-friendly, in addition to being functional. In order to attain this several features that are intuitive and user friendly were introduced.

The schedule interface allows the administrators to see the schedules in a grid format structure. Classes can be rescheduled easily using drag-and-drop functionality, and a preview of conflicts can be enjoyed in real time, before class changes are saved.

Room and teacher management modules are modules that allow the administrators to keep proper records, such as the availability, the room capacity and the workload limitation. These characteristics render the scheduling choices viable and effective.

A. Dashboard Overview

The system gives an administrative summary that will give a concise view of the activities that are to be done in the schedule (the number of classes, rooms, teachers and conflicts identified).

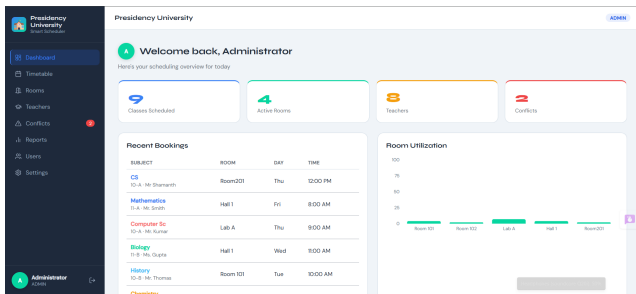


Fig. 6: Administrative Dashboard showing system overview including scheduled classes, active rooms, teachers, and conflict count.

The dashboard will show a list of recent reservations and room occupancy rates as well, and it will allow administrators to keep track of system users effectively.

Additional, like CSV export, printable schedule, analytics dashboards, were also added to facilitate reporting and decision making. Although these features are unnecessary in terms of scheduling, they greatly boost ease of use of the system.

Learning Simple was emphasized during the development to focus on simplicity but without any needless complexity in attracting the user interface.

Copy-Paste: A copy paste is a two state process (NORMAL ↔ COPY MODE), which enables one to copy any slot in any other cell via the API endpoint.

B. Interactive Timetable Interface

The key feature of the system is the interactive timetable interface where the administrators will be able to arrange and operate classes in a visual format.

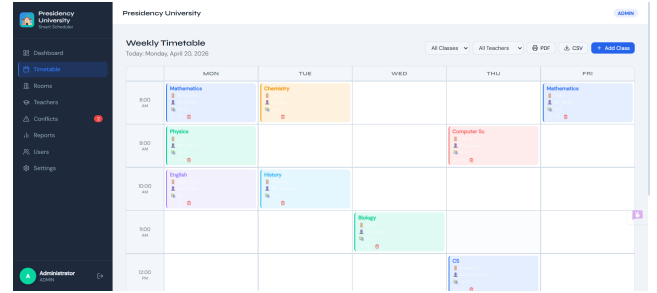


Fig. 7: Weekly timetable interface with drag-and-drop scheduling and real-time conflict visualization.

The interface allows to re-schedule the classes and teachers in real-time, uses performance of the conflict detection and has a class and teacher filter option. All the arranged slots have the subject, room, teacher and the class details.

Real-Time Conflict Preview: When filling in the scheduling form, there is a client-side verification of the desired combinations of teachers, rooms, day, and time-slots with pre-existing data and even yields immediate notifications before it is submitted. This deters any contradictions without the engagement of servers.

C. Conflict Detection Visualization

To eliminate such mistakes as scheduling, the system has real-time conflict detection to avoid booking a room twice, having two different classes of the same teacher, and clashing with other teacher-classes.

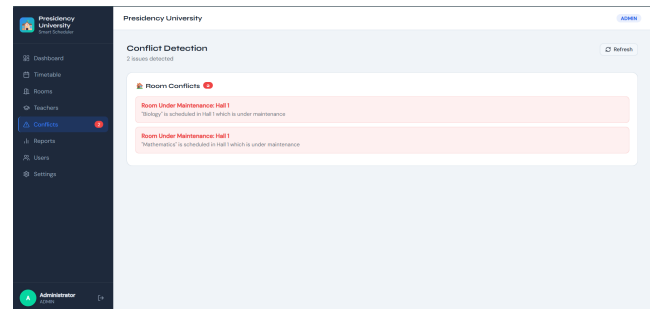


Fig. 8: Conflict detection interface highlighting scheduling conflicts such as overlapping classes and resource clashes.

The system defines various categories of conflicts such as room conflicts, teacher clashes and overlaps of classes. These battles are graphically brought out to enable administrators to rectify matters effectively.

D. Room Management

The system allows completing CRUD of rooms. The qualities of each room are namely the name, capacity, type, which in this case is any of the following: classroom, lab, lecture hall, seminar room, status which could be either active or maintenance, notes and equipment tags.

There is a safeguard to avoid deletion of rooms with classes on. Room utilization is also computed and displayed in the system.

E. Room Management Module

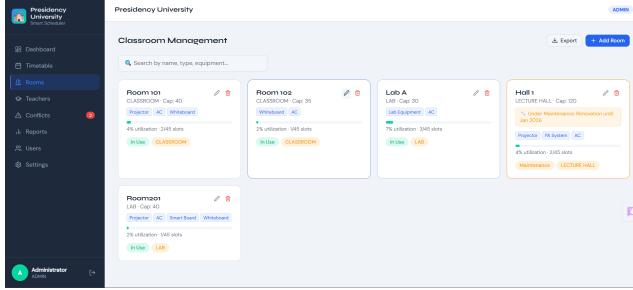


Fig. 9: Room management interface displaying room details, equipment, utilization, and maintenance status.

F. Teacher Management

Teacher profiles contain their contacts, classes that are assigned, maximum workload per week, and their availability per day. The availability breaches and workload overloads are automatically detected by the system.

There is a safety measure such that deleting of teachers who are engaged into classes that are scheduled is not allowed.

G. Teacher Management Module

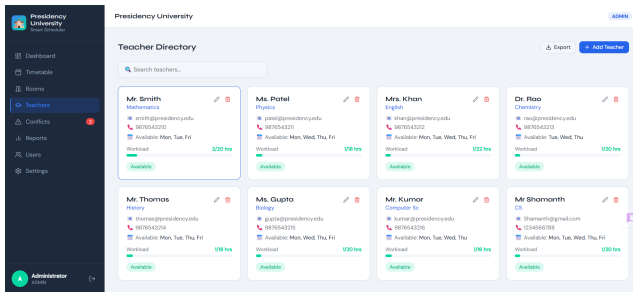


Fig. 10: Teacher directory showing availability, workload tracking, and profile management features.

H. Settings and Configuration

It is possible to setup the system with the help of a special settings page:

- Toggle Saturday: toggles the inclusion or exclusion of the column of Saturdays.
- Custom time slots: enables addition/removal of slots whereby they are automatically sorted.
- Holiday manager: maintains entries on holidays based on dates.

- Blocked slots: it is possible to block certain time intervals (e.g., exams).

I. Reports and Analytics

The reports page offers a variety of different tabular reports (room utilization, teacher workload, class summary) and graphical representations in the form of the charts as well.

The ability to export the timetable, room and teacher information as CSV files. Print option creates a timetable view that can be printed or exported to PDF.

IX. CONCLUSION

The present paper introduces a Smart Classroom and a Timetable Scheduler that can be used in academia in the real world, with an emphasis on the practicality, usability and reliability. As the results show, a constraint-based approach is also capable of dealing with scheduling challenges without utilizing extremely complicated algorithms.

The system focuses on simplicity and transparency which makes it appropriate to be adopted in administrative settings. Future development to improve automation by heuristic methods, becoming more mobile responsive, and scalable to multi-institution deployment are all planned.

Generally, the work has been effective in closing the gap between theory and practice in implementing academic scheduling.

Future Work: Integration of meta-heuristic algorithms like Genetic Algorithms and Simulated Annealing to give rise to conflict-free initial schedules automatically will be improved in the future. Further refinements will be in creating a mobile responsive interface with drag-and-drop capability with a touch interface.

More features are planned such as PDF export via dedicated libraries (e.g., jsPDF), automatic email alerts upon conflict, mass CSV re-import of timetable information and the creation of a student portal features a read-only schedule view. Tenant isolation i.e. multi-institution deployment is also being considered.

ACKNOWLEDGMENT

The authors would wish to acknowledge the Department of Computer Science and Engineering, Presidency University, Bengaluru, as the providers of the infrastructure and guidance of this project. The project guide is given special credits on the grounds of constant guidance, and the academic scheduling staff whose area knowledge was utilized to develop the constraint model.

REFERENCES

- [1] M. C. Chen, S. N. Sze, S. L. Goh, N. R. Sabar, and G. Kendall, "A Survey of University Course timetabling problem Viewpoints, trends, and Opportunities," *IEEE Access*, vol. 9, pp. 106515–106529, 2021. DOI: 10.1109/ACCESS.2021.3100613.
- [2] R. Lewis, survey of metaheuristic-based techniques of university. timetabling problems, *OR Spectrum*, vol. 30, no. 1, pps. 167–190, 2008.
- [3] S. Ceschia, L. Di Gaspero, and A. Schaerf, Design, engineering, and simulated annealing study of a simulated annealing approach to the problem post-enrolment course timetabling problem, *Computers & Operations Research*, vol. 39, no. 7, pp. 1615–1624, 2012.

- [4] E. K. Burke, M. Gendreau, M. Hyde, G. Kendall, G. Ochoa, E. "Ozcan, and R. Qu, "Hyper-heuristics: A survey of the state of the art, *Journal of the Operational Research Society*, vol. 64, no. 12, pp. 1695–1724, 2013.
- [5] Z.-L. Lu and J. K. Hao, adaptive Tabu Search based course timetabling, *European Journal of Operational Research*, vol. 200, no. 1, pp. 235–244, 2010.
- [6] S. L. Goh, G. Kendall and N. R. Sabar, Improved local search. strategies of resolving the post enrolment course timetabling problem, *European Journal of Operations Research*, 261, 1: pp. 17–29, 2017.
- [7] B. McCollum, A perspective on closing the gap between theory and. *Lecture Notes in Computer Science*, practice in university timetabling. vol. 3867, pp. 3–23, 2006.
- [8] R. A. Ranga Prabodanie, "A Model of an Integer Programming of a. Complex University Timetabling Problem: A Case Study, *Industrial Engineering and Management Systems*, 16, 1, pp. 141–153, 2017.
- [9] S. N. Sze, C. L. Bong, K. L. Chiew, W. K. Tiong, and N. A. Bolhassan, University lecture timetabling without pre- Case study. Registration data, in *Proc. Applied IEEE International Conference on Applied. Pp. System Innovation (ICASI)*, 2017. 732–735.
- [10] J. B. Matias, A. C. Fajardo and R. P. Medina, A hybrid genetic course scheduling and teaching workload management algorithm, in *Proc. IEEE HNICEM*, 2018, pp. 1–6, 2019.
- [11] R. a. oude vriel, e. a. jansen, e. w. Hans and j. van Hillegersberg, Higher education institutions practice in timetabling: a systematic. review," *Annals of Operations Research*, vol. 275, no. 1, pp. 145–160, 2019.
- [12] Y. Nagata, at. random partial neighborhood search of the post-enrollment problem of course timetabling, *Computers and Operations Research*, vol. 90, pp. 84–96, 2018.
- [13] S L Goh, G Kendall, N R Sabar and S Abdullah, An effective. combining both methods of searching to find the post enrolment course timetabling. problem," *OPSEARCH*, Jun. 2020.
- [14] E. K. Burke and S. Petrovic, Recent research directions in automated. timetabling, *European Journal of Operational Research*, vol. 140, no. 2, pp. 266–280, 2002.
- [15] M. Lindahl, A. J. Mason, T. Stidsen, and M. Srensen, A strategic view. of University timetabling," *European Journal of Operational Research*, vol. 266, no. 1, pp. 35–45, 2018.
- [16] VMware, Inc., "VMware Spring Boot Reference Documentation, 2023. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- [17] Meta Platforms, Inc., React Documentation, 2023. [Online]. Available: <https://react.dev/>
- [18] Oracle Corporation, "MySQL 8.0 Reference Manual, 2023. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>